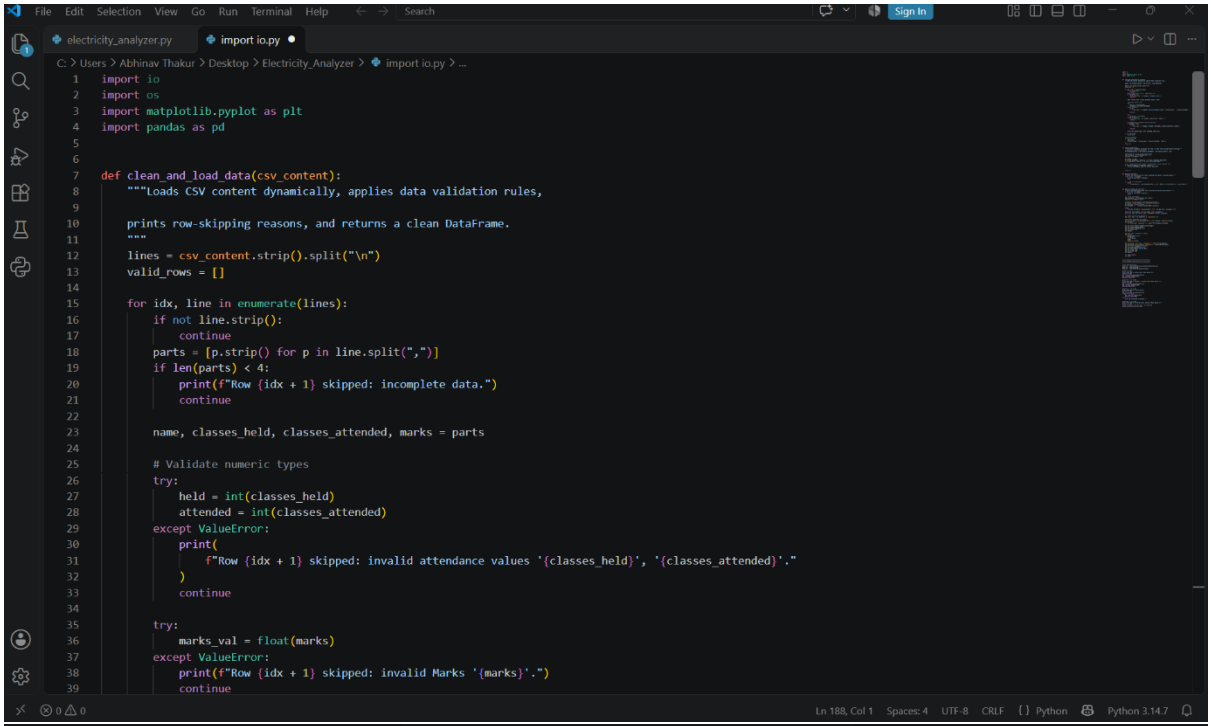


```
import io
import os
import matplotlib.pyplot as plt
import pandas as pd


def clean_and_load_data(csv_content):
    """Loads CSV content dynamically, applies data validation rules,

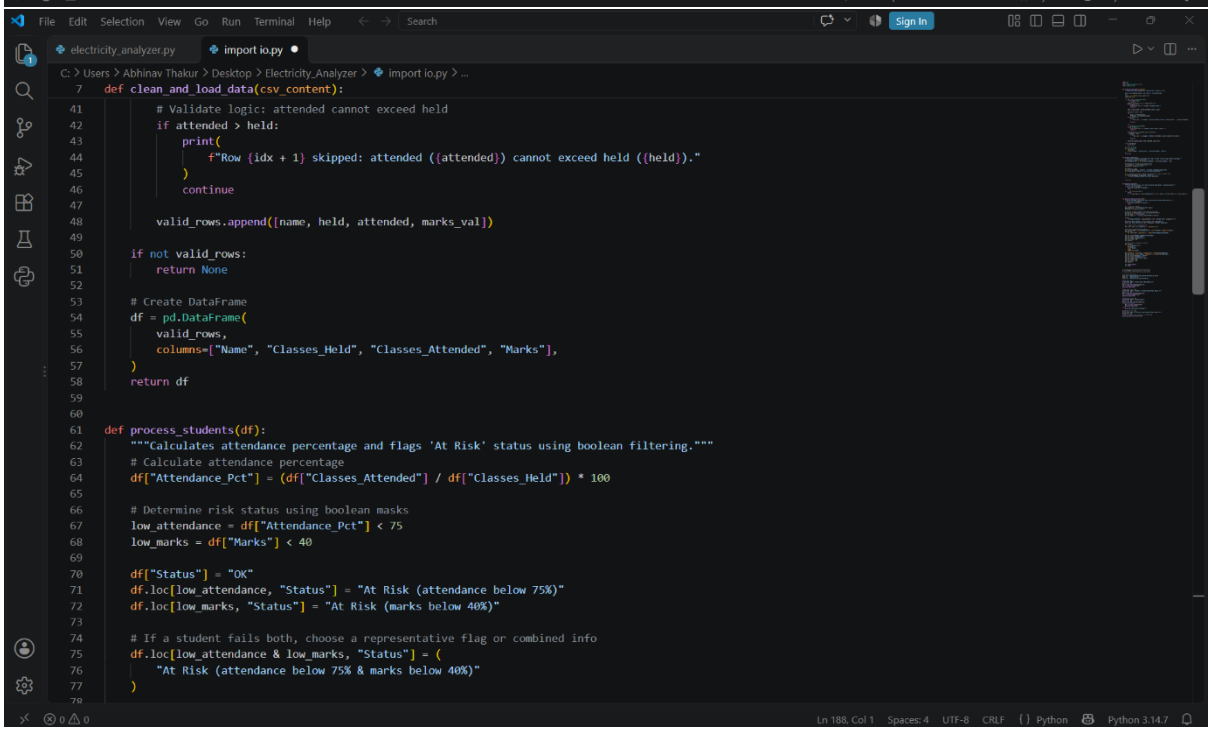
    prints row-skipping reasons, and returns a clean DataFrame.
    """
    lines = csv_content.strip().split("\n")
    valid_rows = []

    for idx, line in enumerate(lines):
        if not line.strip():
            continue
        parts = [p.strip() for p in line.split(",")]
        if len(parts) < 4:
            print(f"Row {idx + 1} skipped: incomplete data.")
            continue

        name, classes_held, classes_attended, marks = parts

        # Validate numeric types
        try:
            held = int(classes_held)
            attended = int(classes_attended)
        except ValueError:
            print(
                f"Row {idx + 1} skipped: invalid attendance values '{classes_held}', '{classes_attended}'."
            )
            continue

        try:
            marks_val = float(marks)
        except ValueError:
            print(f"Row {idx + 1} skipped: invalid Marks '{marks}'.")
            continue
```



```
def clean_and_load_data(csv_content):
            # Validate logic: attended cannot exceed held
            if attended > held:
                print(
                    f"Row {idx + 1} skipped: attended ({attended}) cannot exceed held ({held})."
                )
                continue

            valid_rows.append([name, held, attended, marks_val])

    if not valid_rows:
        return None

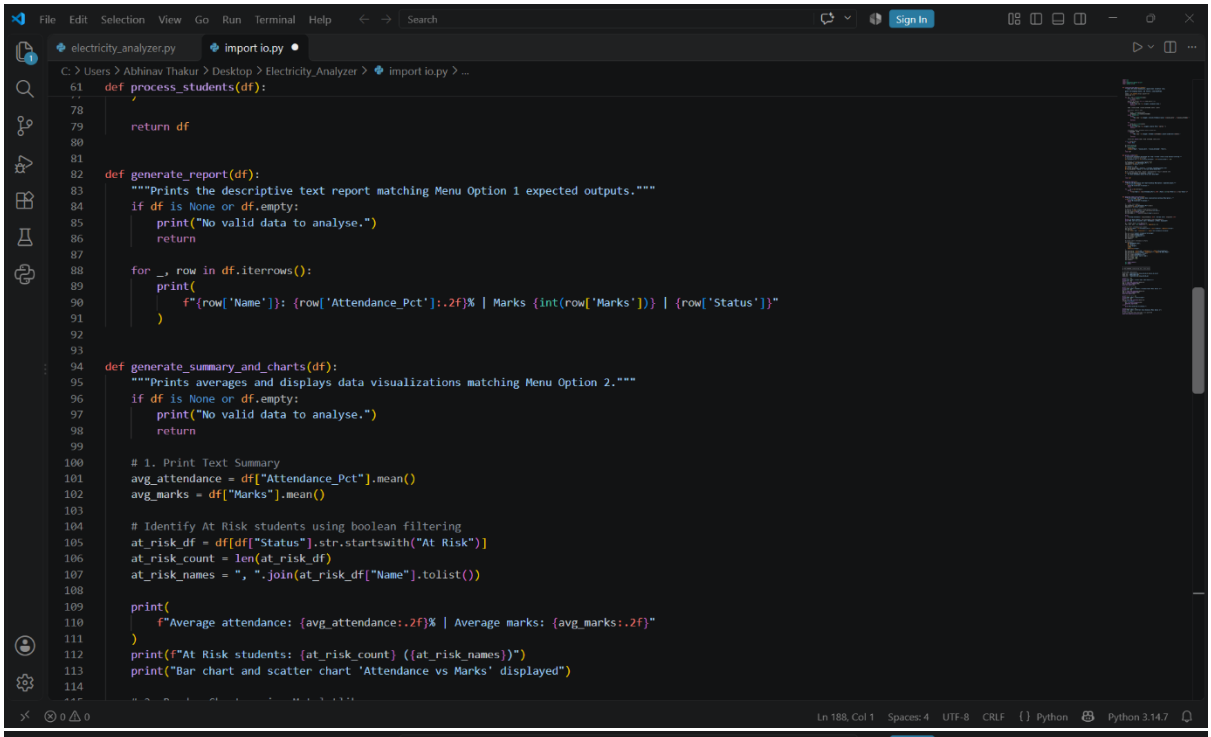
    # Create DataFrame
    df = pd.DataFrame(
        valid_rows,
        columns=["Name", "Classes_Held", "Classes_Attended", "Marks"],
    )
    return df


def process_students(df):
    """Calculates attendance percentage and flags 'At Risk' status using boolean filtering."""
    # Calculate attendance percentage
    df["Attendance_Pct"] = (df["Classes_Attended"] / df["Classes_Held"]) * 100

    # Determine risk status using boolean masks
    low_attendance = df["Attendance_Pct"] < 75
    low_marks = df["Marks"] < 40

    df["Status"] = "OK"
    df.loc[low_attendance, "Status"] = "At Risk (attendance below 75%)"
    df.loc[low_marks, "Status"] = "At Risk (marks below 40%)"

    # If a student fails both, choose a representative flag or combined info
    df.loc[low_attendance & low_marks, "Status"] = (
        "At Risk (attendance below 75% & marks below 40%)"
    )
```



```
def process_students(df):
    return df


def generate_report(df):
    """Prints the descriptive text report matching Menu Option 1 expected outputs."""
    if df is None or df.empty:
        print("No valid data to analyse.")
        return

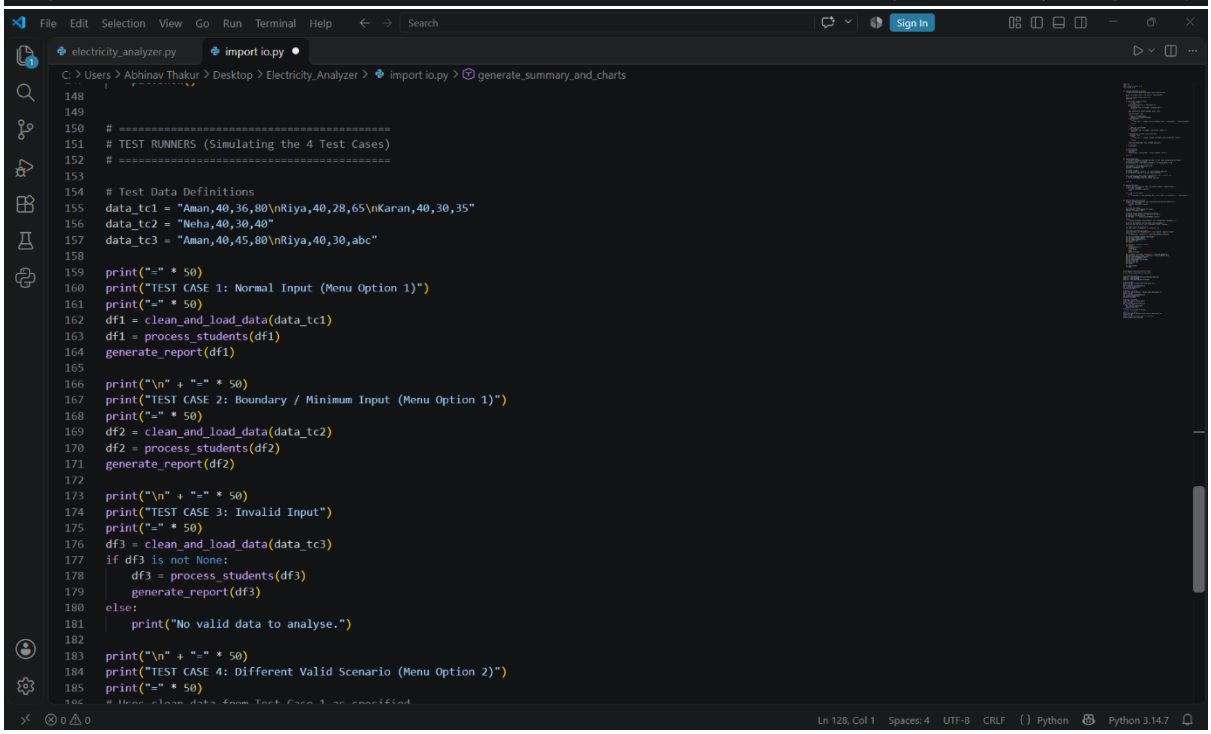
    for _, row in df.iterrows():
        print(
            f"{row['Name']}: {row['Attendance_Pct']:.2f}% | Marks {int(row['Marks'])} | {row['Status']}"
        )


def generate_summary_and_charts(df):
    """Prints averages and displays data visualizations matching Menu Option 2."""
    if df is None or df.empty:
        print("No valid data to analyse.")
        return

    # 1. Print Text Summary
    avg_attendance = df["Attendance_Pct"].mean()
    avg_marks = df["Marks"].mean()

    # Identify At Risk students using boolean filtering
    at_risk_df = df[df["Status"].str.startswith("At Risk")]
    at_risk_count = len(at_risk_df)
    at_risk_names = ", ".join(at_risk_df["Name"].tolist())

    print(
        f"Average attendance: {avg_attendance:.2f}% | Average marks: {avg_marks:.2f}"
    )
    print(f"At Risk students: {at_risk_count} ({at_risk_names})")
    print("Bar chart and scatter chart 'Attendance vs Marks' displayed")
```



```
# ========================================
# TEST RUNNERS (Simulating the 4 Test Cases)
# ========================================

# Test Data Definitions
data_tc1 = "Aman,40,36,80\nRiya,40,28,65\nKaran,40,30,35"
data_tc2 = "Neha,40,30,40"
data_tc3 = "Aman,40,45,80\nRiya,40,30,abc"

print("=" * 50)
print("TEST CASE 1: Normal Input (Menu Option 1)")
print("=" * 50)
df1 = clean_and_load_data(data_tc1)
df1 = process_students(df1)
generate_report(df1)

print("\n" + "=" * 50)
print("TEST CASE 2: Boundary / Minimum Input (Menu Option 1)")
print("=" * 50)
df2 = clean_and_load_data(data_tc2)
df2 = process_students(df2)
generate_report(df2)

print("\n" + "=" * 50)
print("TEST CASE 3: Invalid Input")
print("=" * 50)
df3 = clean_and_load_data(data_tc3)
if df3 is not None:
    df3 = process_students(df3)
    generate_report(df3)
else:
    print("No valid data to analyse.")

print("\n" + "=" * 50)
print("TEST CASE 4: Different Valid Scenario (Menu Option 2)")
print("=" * 50)
```

File Edit Selection View Go Run Terminal Help Search Sign In

electricity_analyzer.py | import io.py ●

C: > Users > Abhinav Thakur > Desktop > Electricity_Analyzer > import io.py > generate_summary_and_charts

```
155  data_tc1 = "Aman,40,36,80\nRiya,40,28,65\nKaran,40,30,35"
156  data_tc2 = "Neha,40,30,40"
157  data_tc3 = "Aman,40,45,80\nRiya,40,30,abc"
158
159  print("=" * 50)
160  print("TEST CASE 1: Normal Input (Menu Option 1)")
161  print("=" * 50)
162  df1 = clean_and_load_data(data_tc1)
163  df1 = process_students(df1)
164  generate_report(df1)
165
166  print("\n" + "=" * 50)
167  print("TEST CASE 2: Boundary / Minimum Input (Menu Option 1)")
168  print("=" * 50)
169  df2 = clean_and_load_data(data_tc2)
170  df2 = process_students(df2)
171  generate_report(df2)
172
173  print("\n" + "=" * 50)
174  print("TEST CASE 3: Invalid Input")
175  print("=" * 50)
176  df3 = clean_and_load_data(data_tc3)
177  if df3 is not None:
178      df3 = process_students(df3)
179      generate_report(df3)
180  else:
181      print("No valid data to analyse.")
182
183  print("\n" + "=" * 50)
184  print("TEST CASE 4: Different Valid Scenario (Menu Option 2)")
185  print("=" * 50)
186  # Uses clean data from Test Case 1 as specified
187  generate_summary_and_charts(df1)
188
```

File Edit Selection View Go Run Terminal Help Search Sign In

Problems Output Debug Console Terminal Ports

Python Python Python

```
PS C:\Users\Abhinav Thakur\AppData\Local\Programs\Microsoft VS Code> & "C:\Users\Abhinav Thakur\AppData\Local\Python\pythoncore-3.14-64\python.exe" "c:/Users/Abhinav Thakur/Desktop/Electricity_Analyzer/import io.py"
==================================================
TEST CASE 1: Normal Input (Menu Option 1)
==================================================
Aman: 90.00% | Marks 80 | OK
Riya: 70.00% | Marks 65 | At Risk (attendance below 75%)
Karan: 75.00% | Marks 35 | At Risk (marks below 40%)

==================================================
TEST CASE 2: Boundary / Minimum Input (Menu Option 1)
==================================================
Neha: 75.00% | Marks 40 | OK

==================================================
TEST CASE 3: Invalid Input
==================================================
Row 1 skipped: attended (45) cannot exceed held (40).
Row 2 skipped: invalid Marks 'abc'.
No valid data to analyse.

==================================================
TEST CASE 4: Different Valid Scenario (Menu Option 2)
==================================================
Average attendance: 78.33% | Average marks: 60.00
At Risk students: 2 (Riya, Karan)
Bar chart and scatter chart 'Attendance vs Marks' displayed
```

